

บทที่ 5

การพัฒนาดาต้าแวร์เฮ้าส์เพื่อนำไปใช้งาน

จากบทที่ผ่านมา เราได้ศึกษาถึงวิธีการออกแบบดาต้าแวร์เฮ้าส์โดยใช้หลักการของระบบฐานข้อมูลเชิงวัตถุสัมพันธ์ สำหรับในบทนี้เราจะกล่าวถึงการพัฒนาระบบเพื่อนำไปใช้งานจริง

สำหรับเครื่องมือที่ใช้ในการพัฒนาระบบ เราจะใช้ IBM DB2 ซึ่งเป็นเครื่องมือที่ถูกพัฒนาโดยบริษัท ไอบีเอ็ม จำกัด ซึ่งมีอุปกรณ์ที่จำเป็นต่อการพัฒนาระบบฐานข้อมูลเชิงวัตถุสัมพันธ์

สาเหตุประการสำคัญที่ใช้ IBM DB2 ในการพัฒนาดาต้าแวร์เฮ้าส์นั้นคือการที่โปรแกรมดังกล่าวสามารถรองรับการทำงานของระบบฐานข้อมูลเชิงวัตถุสัมพันธ์ และสามารถทำให้โปรเจกต์นี้สามารถทำได้ นอกจากนี้บริษัทไอบีเอ็มยังมีคู่มือการใช้งานสามารถให้ดาวน์โหลดได้จากเว็บไซต์ของบริษัทอีกด้วย

ในบทนี้ เริ่มต้นเราจะกล่าวถึงภาษาโครงสร้างข้อมูล (DDL : Data Definition Language) ในหัวข้อ 5.1 ซึ่งภาษานี้จะรวบรวมคำสั่งที่ใช้ในการสร้างดาต้าแวร์เฮ้าส์ขึ้นมา ในหัวข้อ 5.2 นั้นเราจะแสดงภาษาในการจัดการฐานข้อมูล ซึ่งก็จะเกี่ยวข้องกับคำสั่งในการแทรกข้อมูล (Insert) , ลบข้อมูล (Delete) และอัปเดตข้อมูล (Update) และในหัวข้อ 5.3 ซึ่งเป็นหัวข้อสุดท้ายเราจะแสดงให้เห็นถึงการตอบคำถามเพื่อใช้ในกระบวนการทางธุรกิจ สำหรับภาษาที่ใช้นั้นเราจะใช้เป็น เอสคิวแอล 99 (SQL 99) และเพื่อให้ง่ายต่อการอ่าน สำหรับคำสั่งของเอสคิวแอลเราจะใช้เป็นสัญลักษณ์ตัวพิมพ์ใหญ่

5.1 คำสั่งที่ใช้ในการสร้างออบเจกต์และตาราง

สำหรับการสร้างฐานข้อมูลของดาต้าแวร์เฮ้าส์นั้น เราจะสร้างจากแผนภาพอีอาร์ในบทที่ผ่านมา และเพื่อให้ง่ายเราจะทำการแปลงชนิดของเอนทิตีและแอททริบิวต์จากแผนภาพอีอาร์มายังตาราง

สำหรับในหัวข้อนี้เราจะเริ่มจากสร้างตารางโดเมนชั้นง่าย ๆ ได้แก่ โดเมนชั้นของเวลา(Time Dimension)และ โดเมนชั้นของสินค้า(Product Dimension)ก่อน หลังจากนั้นก็จะทำการสร้างตารางโดเมนชั้นที่ซับซ้อนกว่าคือ โดเมนชั้นของลูกค้า(Customer Dimension) และสุดท้ายเราก็จะสร้างตารางแฟกต์(Fact Table)

5.1.1 ตารางโดเมนชั้นของเวลา(Time Dimension Table)

สำหรับตารางโดเมนชั้นของเวลา และ ออบเจกต์ของโดเมนชั้นของเวลา(Time Dimension Object) นั้นประกอบด้วย ซิมเปิลแอททริบิวต์(Simple Attribute) ได้แก่ แอททริบิวต์ “oid” เป็นแอททริบิวต์ที่ทำหน้าที่เป็นคีย์แอททริบิวต์ในส่วนนี้ โดยปกติจะไม่แสดงให้เห็นเพราะจะถูกซ่อนอยู่ในตาราง แอททริบิวต์ “Time_Key” ในส่วนนี้จะมีหรือไม่ก็ได้ สำหรับในฐานข้อมูลนี้จะให้มีเพื่อให้ง่ายต่อการอ้างอิงข้อมูลในตาราง โดเมนชั้นของเวลา แอททริบิวต์“Fiscal_year” , “Fiscal_Qtr” และ “Fiscal_month” จะเป็นค่าที่ถูกเก็บลงในตารางโดเมนชั้นของเวลา โดยที่แอททริบิวต์ทั้งสามตัวจะถูกอ้างอิงถึงโดย Time_Key เช่น หาก Time_Key มีค่าเท่ากับ “2002Q3M09” ก็จะอ้างอิงถึงแถวที่มี Fiscal_year เท่ากับ 2002 Fiscal_Qtr เท่ากับ Quarter3 และ มี Fiscal_month เท่ากับ September เป็นต้น

แต่ละแถวใน ตารางโดเมนชั้นของเวลา นั้น สามารถอ้างอิงได้จากตารางอื่น ๆ เช่น หากตาราง Sale_Fact_Table ต้องการอ้างอิงข้อมูลในตารางโดเมนชั้นของเวลา โดยการใช้คำสั่ง REF data-type ใน

ระบบฐานข้อมูลเชิงสัมพันธ์ สำหรับระบบฐานข้อมูลเชิงวัตถุสัมพันธ์นี้เราต้องสร้างโครงสร้างของออปเจก (Object Type) ก่อน โดยใช้คำสั่งเอสคิวแอล 99 ดังนี้

```
CREATE TYPE time_dimension_t AS
(
    time_key          CHAR(10),
    fiscal_year       NUM(4),
    fiscal_qtr        CHAR(10),
    fiscal_month      CHAR(10)
)
MODE DB2SQL
REF USING INTEGER
```

สำหรับประเภท (data-type) ของแอทริบิวต์ของออปเจกนั้นจะถูกประกาศในโครงสร้างข้อมูลของออปเจก และเมื่อเราประกาศ Object data-type แล้วเราก็สามารถนำไปประกาศเป็นโครงสร้างข้อมูลใหม่ หรือนำไปสร้างเป็นตารางได้แล้ว สำหรับคำสั่ง REF USING INTEGER นั้นเป็นการระบุให้ Oid ที่นำมาใช้ในการอ้างถึงโครงสร้างข้อมูลชนิดนี้ให้เป็นตัวเลข และสำหรับข้อกำหนดต่าง ๆ ของแอทริบิวต์นั้นจะถูกประกาศในขณะที่ทำการสร้างตาราง

สำหรับตารางใดเม้นชันของเวลาจะใช้ชื่อ time_dim_tab ซึ่งจะถูกประกาศโดยใช้คำสั่ง ดังนี้

```
CREATE TABLE time_dim_tab OF time_dimension_t
(
    REF IS OID USER GENERATED,
    time_key WITH OPTION NOT NULL,
    fiscal_qtr WITH OPTION NOT NULL,
    fiscal_month WITH OPTION NOT NULL,
    fiscal_year WITH OPTION NOT NULL
)
```

แต่ละแถวที่ถูกสร้างขึ้นในตาราง time_dim_tab นั้น จะเป็น time_dimension_t ซึ่งเป็นออปเจกที่ถูกสร้างขึ้น จึงเรียกประเภทของตารางนี้ว่า ออปเจกเทเบิล (Object Table) นอกจากนี้แต่ละแถวยังสามารถที่จะอ้างถึงได้จากออปเจกอื่น ๆ ซึ่งเราจะกล่าวถึงในภายหลัง

แต่ละแถวจะมี ออปเจกไอดีคอนทิไฟเลอร์ (OID) ซึ่งจะใช้อ้างถึงข้อมูลในแต่ละแถว แต่ใน IBM DB2 นั้น ไม่สามารถให้ระบบสร้าง oid ได้ เราจึงต้องสร้าง ลำดับ(Sequence) และ ทรigger(Trigger)มาเพื่อ

แก้ไข สำหรับการสร้าง oid นั้นเราจะใช้คำสั่ง “REF IS OID USER GENERATED” สำหรับรูปที่ 5-1 จะแสดงให้เห็นถึงคอลัมน์ต่าง ๆ ของ time_dim_tab

Table time_dim_tab (of time_dimension_t)			
Oid	Fiscal_year	Fiscal_Qtr	Fiscal_month
Integer	Number	Char(10)	Char(10)
Object Identifier	NOT NUL	NOT NULL	NOT NULL

รูปที่ 5-1 Object – Relational Representation of the Time Dimension Table

5.1.2 ตารางไคเมนชันของสินค้า(Product Dimension Table)

คล้ายกับตารางไคเมนชันของเวลา สำหรับตารางไคเมนชันของสินค้า นั้นจะใช้ชื่อว่า Prod_dim_tab โดยสำหรับตารางนี้จะประกอบด้วย ซิมเปิลแอททริบิวต์(Simple Attribute) ได้แก่ oid ซึ่งทำหน้าที่เป็นออปเจ็คไคเดนทิไฟเลอร์ ที่ชี้ไปยังแต่ละแถวของตาราง Prod_dim_tab นอกจากนี้ยังมีแอททริบิวต์อื่น ๆ ได้แก่ model_name , manufacturer, type, engine_size และก่อนที่จะสร้าง Prod_dim_tab นั้น เราจำเป็นต้องสร้าง prod_dimension_t ซึ่งเป็นโครงสร้างของข้อมูลขึ้นมาก่อน โดยสำหรับคำสั่งเอสคิวแอลที่ใช้ในการสร้าง Prod_dimention_t คือ

```
CREATE TYPE prod_dimension_t AS
(
    model_name    VARCHAR(30),
    manufacturer  VARCHAR(25),
    type          VARCHAR(50),
    engine_size   CHAR(6)
)
MODE DB2SQL
REF USING INTEGER
```

และเมื่อสร้าง Prod_dimension_t ซึ่งเป็นโครงสร้างของข้อมูลขึ้นมาแล้วเราก็นำโครงสร้างของข้อมูลดังกล่าวมาสร้างเป็นตาราง ตามคำสั่งเอสคิวแอลดังนี้

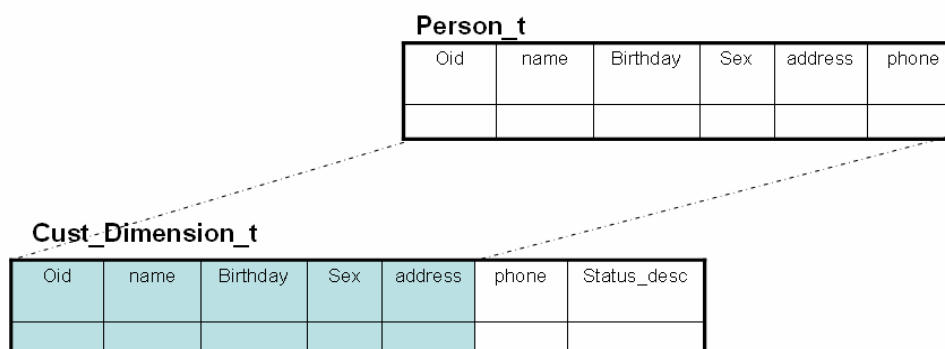
```
CREATE TABLE prod_dim_tab OF prod_dimension_t
(
    REF IS OID USER GENERATED,
    manufacturer WITH OPTION NOT NULL,
    model_name WITH OPTION NOT NULL,
    type WITH OPTION NOT NULL,
    engine_size WITH OPTION NOT NULL
)
```

Table Prod_dim_tab (of prod_dimension_t)				
Oid	Manufacturer	Model_name	type	Engine_size
Integer	VARCHAR(25)	VARCHAR(30)	VARCHAR(50)	Char(6)
Object Identifier	NOT NUL	NOT NULL	NOT NULL	NOT NULL

รูปที่ 5-2 Object – Relational Representation of the Product Dimension Table

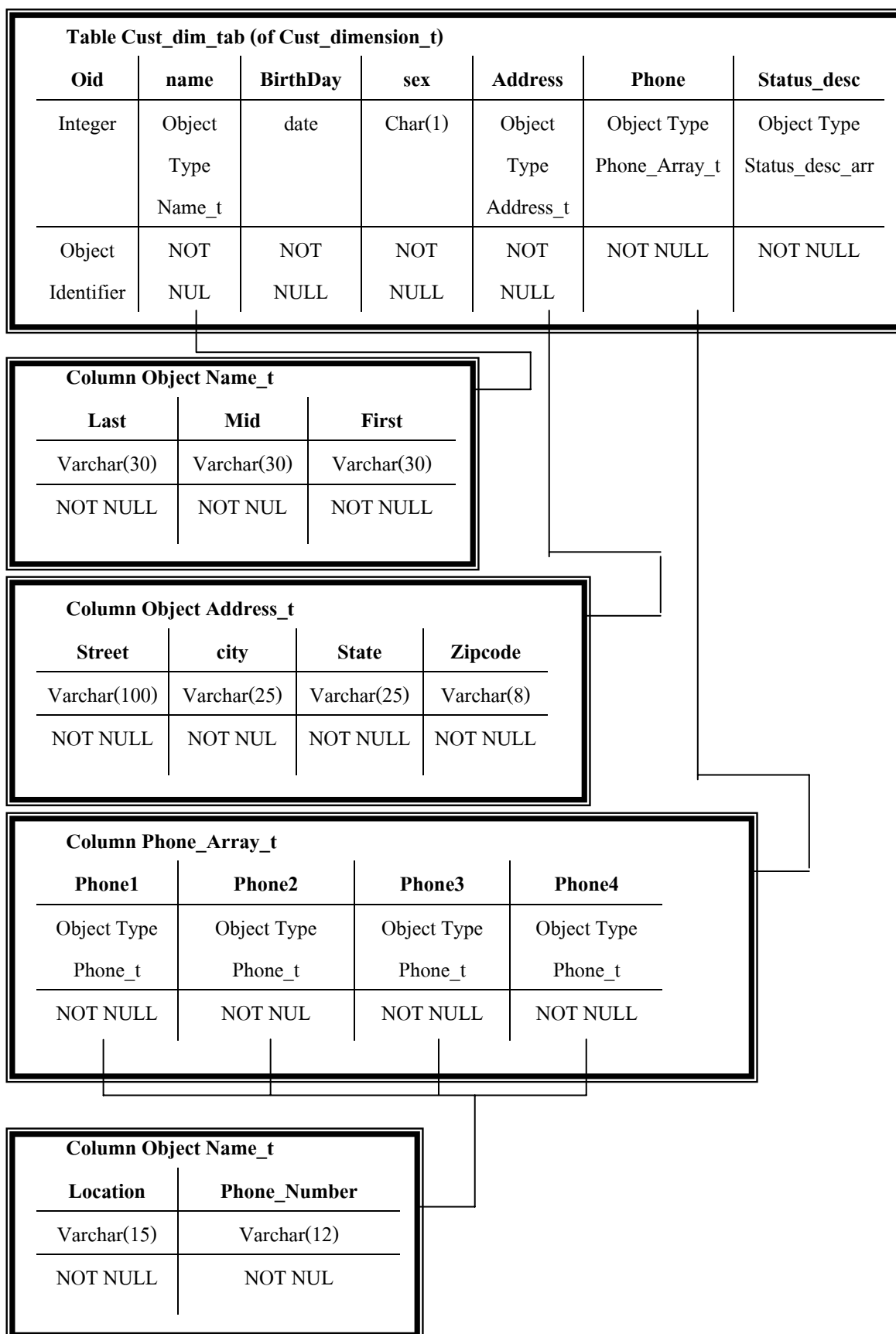
5.1.3 ตารางไคเมนชันของลูกค้า

สำหรับตารางไคเมนชันของลูกค้าและออปเจ็กต์ไคเมนชันของลูกค้านั้นจะแตกต่างจากตารางไคเมนชันและออปเจ็กต์ไคเมนชันที่ผ่านมา ตรงที่โครงสร้างของไคเมนชันของลูกค้านั้นสร้างมาจากโครงสร้างของข้อมูลที่ชื่อ Cust_dimension_t ซึ่ง Cust_dimension_t นั้นจะมีโครงสร้างหลัก(Root Type) เป็น Person_t หมายความว่า Cust_dimension_t นั้นได้รับการถ่ายทอด (Inherit) คุณสมบัติ(Attribute) มาจาก Person_t ดังแสดงให้เห็นได้ในรูปที่ 5-3

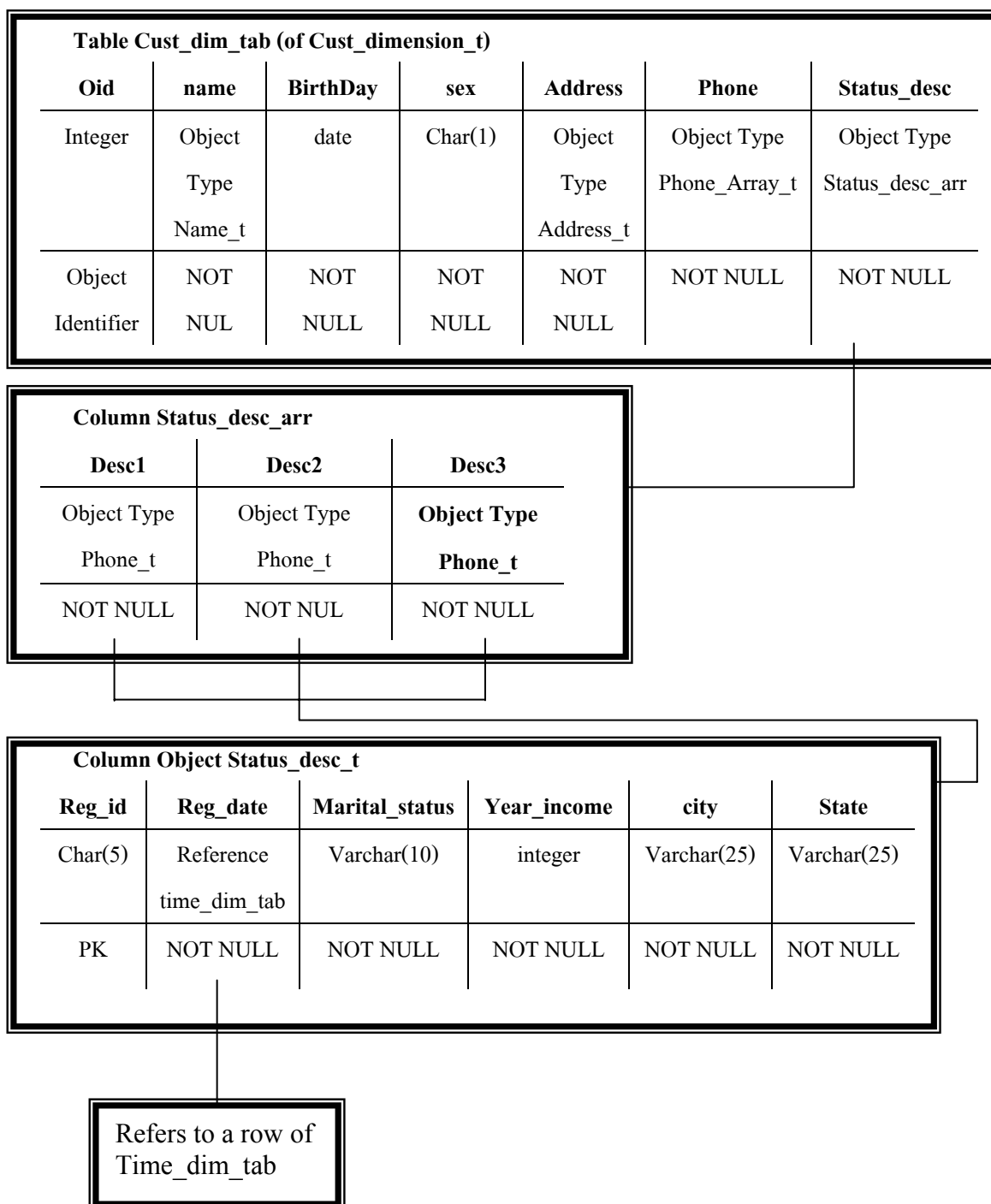


รูปที่ 5-3 แสดงการถ่ายทอดคุณสมบัติระหว่าง Person_t กับ Cust_dimension_t

สำหรับไคเมนชันนี้มีทั้ง ซิมเปิลแอททริบิวต์(Simple Attribute),คอมโพสิตแอททริบิวต์(Composite Attribute) และมัลติแวลูแอททริบิวต์ (Multivalues Attribute) ดังแสดงให้เห็นในรูปที่ 5-4



รูปที่ 5-4 n Object – Relational Representation of the Customer Dimension Table



รูปที่ 5-4 ข Object – Relational Representation of the Customer Dimension Table(Continue)

สำหรับ IBM DB2 นั้นไม่สามารถที่จะประกาศประเภทของแอทริบิวต์เป็น โครงสร้างซับซ้อนได้ ดังนั้นจึงทำให้เราต้องสร้างโครงสร้างข้อมูลชนิด Array ขึ้นมารองรับความสามารถของมัลติแวลูแอทริบิวต์ ดังรูปข้างต้น ต่อไปนี้เราจะแสดงคำสั่งในการสร้างโครงสร้างข้อมูล โดยเริ่มจากการประกาศ คอมโพสิสไทป์ก่อน

```

CREATE TYPE name_t AS
(
    First VARCHAR(20),
    Middle VARCHAR(15),
    Last VARCHAR(20)
)
MODE DB2SQL

CREATE TYPE Address_t AS
(
    No INTEGER,
    Street VARCHAR(100),
    City VARCHAR (25),
    State VARCHAR (25),
    Zipcode VARCHAR (25)
)
MODE DB2SQL

```

รูปที่ 5-5 แสดงคำสั่งเอสคิวแอลในการสร้างคอมโพสิสไทป์

ต่อมาเราก็จะแสดงคำสั่งในการสร้างมัลติแวลูแอททริบิวต์

```

CREATE TYPE phone_t AS
(
    location VARCHAR(15),
    phone_num VARCHAR(12)
)
MODE DB2SQL

```

รูปที่ 5-6ก แสดงคำสั่งเอสคิวแอลในการสร้างมัลติแวลูไทป์

```
CREATE TYPE phone_array_t AS
```

```
(
    phone1 PHONE_T,
    phone2 PHONE_T,
    phone3 PHONE_T,
    phone4 PHONE_T
)
```

```
MODE DB2SQL
```

```
REF USING INTEGER
```

```
CREATE TYPE status_desc_t AS
```

```
(
    Reg_id          CHAR(5),
    reg_date        REF(time_dimension_t),
    marital_status   VARCHAR(10),
    year_income      INTEGER,
    city            VARCHAR(25),
    state           VARCHAR(25)
)
```

```
MODE DB2SQL
```

```
CREATE TYPE status_desc_arr AS
```

```
(
    desc1    status_desc_t,
    desc2    status_desc_t,
    desc3    status_desc_t
)
```

```
MODE DB2SQL
```

```
REF USING INTEGER
```

รูปที่ 5-6x แสดงคำสั่งเอสคิวแอลในการสร้างมัลติแวลูไทป์(ต่อ)

ต่อมาเราก็จะสร้างโครงสร้างหลัก ซึ่งก็คือ Person_t

```
CREATE TYPE person_t AS
(
    name          name_t,
    birthday      DATE,
    sex           CHAR(1),
    address       address_t,
    phone         phone_array_t
)
MODE DB2SQL
REF USING INTEGER
NOT FINAL
```

และสุดท้ายเราจะสร้าง Cust_dimension_t ที่ได้รับการถ่ายทอด มาจาก Person_t

```
CREATE TYPE cust_dimension_t UNDER person_t AS
(
    status_desc   status_desc_arr
)
MODE DB2SQL
```

เมื่อสร้างโครงสร้างข้อมูลทั้งหมดเรียบร้อยแล้วขั้นตอนต่อไปคือการสร้างตารางไดเมนชันของ
ลูกค้าที่ชื่อ Cust_dimension_tab เราจะใช้คำสั่งเอสคิวแอลในการสร้าง Cust_dimension_tab ดังนี้

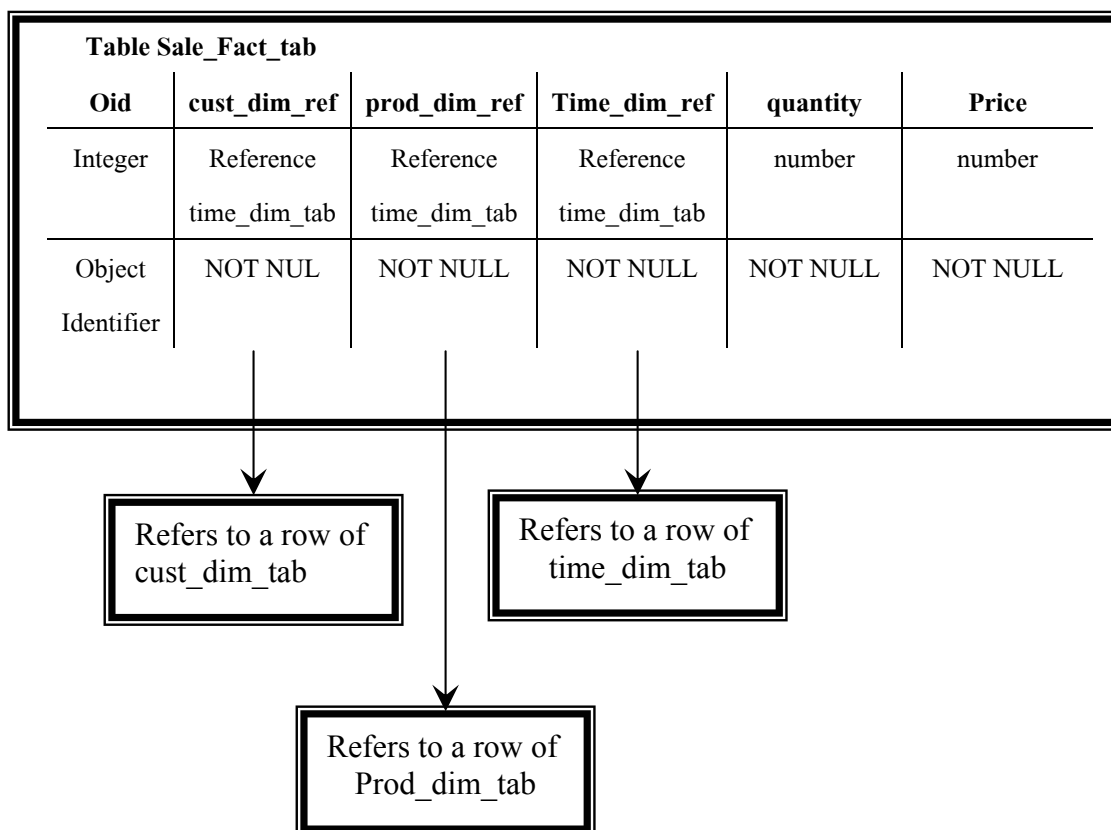
```
CREATE TABLE cust_dim_tab OF cust_dimension_t
(
    REF IS OID USER GENERATED,
    name          WITH OPTIONS NOT NULL,
    birthday      WITH OPTIONS NOT NULL,
    sex           WITH OPTIONS NOT NULL,
    address       WITH OPTIONS NOT NULL,
    phone         WITH OPTIONS NOT NULL,
```

status_desc WITH OPTIONS NOT NULL

)

5.1.4 ตารางแสดงข้อมูลการขายสินค้า(Sale Fact Table)

สำหรับตารางแฟคที่อยู่ตรงกลางของไดอะแกรมนั้น จะมีโครงสร้างง่าย ๆ ดังรูปที่ 5-7



รูปที่ 5-7 Object – Relational Representation of the Sale Fact Table

เหมือนกับตารางอื่น ๆ ก็จะต้องสร้างโครงสร้างของออปเจ็กต์ ขึ้นมาก่อน ในที่นี้ก็คือ sale_fact_t ซึ่งในตารางแฟค นั้นจะมีแอททริบิวต์อยู่ 3 ตัวที่ทำหน้าที่เป็นพอยเตอร์ชี้ไปยังตารางอื่น ๆ ซึ่งทั้ง 3 ตัวนี้จะต้องมีทุกครั้งที่มีการเพิ่มแถวเข้าไปยังตาราง สำหรับคำสั่งที่ใช้ในการสร้าง Sale_Fact_tab นั้นแสดงได้ดังข้างล่างนี้

```
CREATE TYPE sale_fact_t AS
(
  cust_dim      REF(cust_dimension_t),
  product_dim   REF(prod_dimension_t),
  time_dim      REF(time_dimension_t),
  quantity      NUM,
  price         NUM
)
```

```
MODE DB2SQL
REF USING INTEGER
```

```
CREATE TABLE sale_fact_tab OF sale_fact_t
(
  REF IS OID USER GENERATED,
  cust_dim WITH OPTION NOT NULL,
  product_dim WITH OPTION NOT NULL,
  time_dim WITH OPTION NOT NULL,
  quantity WITH OPTION NOT NULL,
  price WITH OPTION NOT NULL
)
```

และในตาราง Sale_fact_tab นั้นมีแอททริบิวต์ 3 ตัวที่ทำหน้าที่เป็นพอยเตอร์ดังนั้นจึงต้องทำการ Add Scope เพื่อให้แอททริบิวต์ทั้ง 3 นั้นสามารถชี้ไปได้ถูกที่ สำหรับคำสั่งที่ใช้ในการเพิ่มมุมมอง(Add Scope)นั้นจะใช้คำสั่ง ALTER TABLE ดังคำสั่งข้างล่างนี้

```
ALTER TABLE sale_fact_tab ALTER COLUMN time_dim ADD SCOPE time_dim_tab
ALTER TABLE sale_fact_tab ALTER COLUMN product_dim ADD SCOPE prod_dim_tab
ALTER TABLE sale_fact_tab ALTER COLUMN cust_dim ADD SCOPE cust_dim_tab
```

5.2 คำสั่งที่ใช้ในการเปลี่ยนแปลงข้อมูล

โดยปกติแล้ว ดาต้าแวร์เฮ้าส์นั้นจะเก็บข้อมูลชนิดอ่านอย่างเดียว การเก็บข้อมูลลงดาต้าแวร์เฮ้าส์นั้นไม่จำเป็นมากนัก แต่อย่างไรก็ตามหากแอททริบิวต์ในตารางถูกเปลี่ยน ข้อมูลเก่าก็จะถูกแทนที่ด้วยข้อมูลใหม่ที่เข้ามา หรือข้อมูลใหม่อาจจะถูกแทรกลงตารางโดยที่ข้อมูลเก่าไม่ได้ถูกเปลี่ยนแปลง สำหรับการลบข้อมูลนั้นแทบจะไม่จำเป็นในการใช้งานเลย เพราะว่าที่เก็บข้อมูล (Data Storage) มีราคาสูง แต่ถึงอย่างไรก็ตามการอัปเดต การลบ และการแทรกข้อมูลลงดาต้าแวร์เฮ้าส์ก็ยังคงต้องมีเพราะยังคงมีการเปลี่ยนแปลงข้อมูลในแต่ละโดเมนชั้นอยู่ สำหรับในหัวข้อนี้เราจะกล่าวถึง การแทรก การลบ และการอัปเดตข้อมูล

ในหัวข้อที่ผ่านมาเราแบ่งออกเป็น 4 หัวข้อย่อย แต่ละหัวข้อย่อยก็จะกล่าวถึงแต่ละตาราง สำหรับในหัวข้อนี้เราจะพูดถึงภาษาที่ใช้ในการเปลี่ยนแปลงข้อมูลไปที่ตาราง

5.2.1 ตารางไคเมนชันของเวลา

สำหรับคำสั่งที่ใช้ในการแทรกข้อมูลลงในตาราง time_dim_tab นั้น จะแสดงให้เห็นดังข้างล่างนี้

```
INSERT INTO time_dim_tab VALUES (
time_dimension_t(0),'1999Q1M01',1999,'Quarter1','January');
```

การอัปเดตข้อมูลสามารถทำได้โดยการใช้คำสั่ง

```
UPDATE time_dim_tab
SET fiscal_year = 1998,
fiscal_qtr='Quarter2',
fiscal_month = 'April'
WHERE time_key = '1999Q1M01'
```

การลบข้อมูลสามารถทำได้โดยการใช้คำสั่ง

```
DELETE FROM time_dim_tab
WHERE time_key = '1999Q1M01'
```

จากการศึกษาเราจะพบว่า คีย์ที่เรานำมาใช้ในการอ้างอิง (time_key : อาจจะมีหรือไม่ก็ได้เพราะยังไม่มี OID อยู่แล้ว แต่ในที่นี้ให้มีเพราะเพื่อให้ง่ายต่อการอ้างอิง) นั้นเป็นคีย์ที่เป็นลักษณะคีย์ที่มีความหมาย (Meaningful values) ข้อดีของการใช้คีย์ที่มีความหมายคือการที่ไม่ต้องประยุกต์ใช้ในการตอบคำถาม แต่ในทางปฏิบัติแล้วการใช้คีย์ที่มีความหมายเป็นสิ่งที่ควรหลีกเลี่ยงเพราะหากต้องมีการเปลี่ยนแปลงค่าแล้ว จะทำให้คีย์จะต้องถูกเปลี่ยนด้วยซึ่งเป็นสิ่งที่ไม่สมควรทำอย่างยิ่ง

5.2.2 ตารางไคเมนชันของสินค้า

สำหรับคำสั่งในการแทรก ลบ หรืออัปเดตข้อมูลใน prod_dim_tab นั้นจะมีลักษณะง่าย ๆ คล้ายกับคำสั่งที่ใช้ในการแทรก ลบ หรืออัปเดตข้อมูลใน time_dim_tab ซึ่งได้กล่าวไปแล้วข้างต้น สำหรับคำสั่งในการแทรกข้อมูลลงใน prod_dim_tab นั้นแสดงได้ดังนี้

```
INSERT INTO prod_dim_tab VALUES(
prod_dimension_t(0),
'Corolla',
'Toyota',
'CE 4dr Seden(4cyl,4A)',
'1800CC')
```

ข้อมูลสามารถอัปเดตได้โดยการใช้คำสั่งดังนี้

```
UPDATE prod_dim_tab
SET model_name = 'Civic',
Type = '2 Door Sedan',
Engine_size = '500CC'
WHERE model_name = 'Corolla'
```

คำสั่งข้างล่างนี้เป็นคำสั่งที่ใช้ในการลบ

```
DELETE FROM prod_dim_tab
WHERE model_name = 'Civic', Type = '2 Door Sedan'
```

5.2.3 ตารางไคเมนชันของลูกค้า

สำหรับไคเมนชันนี้จะแตกต่างจากไคเมนชันอื่น เนื่องจากไคเมนชันของลูกค้านี้จะประกอบไปด้วยข้อมูลประเภทโครงสร้างซึ่งได้แก่ข้อมูลประเภท คอมโพสิสแอททริบิวต์ , มัลติแวลูแอททริบิวต์ ซึ่งภาษาที่ใช้ในการเปลี่ยนแปลงข้อมูลในดาต้าแวร์เฮ้าส์ก็จะมีลักษณะที่แตกต่างออกไป ดังตัวอย่างข้างล่างนี้ ก็จะเป็นการแทรกข้อมูลลงในตารางไคเมนชันของลูกค้า ที่ชื่อ cust_dim_tab

```
INSERT INTO cust_dim_tab VALUES (
cust_dimension_t(0),
name_t()..First('Jonathan')..Middle('-')..Last('Abbott'),
'12/11/1999',
'M',
Address_t()..Street('142 S Sheridan Road')..City('Tulsa')..State('Oklahoma')..Zipcode('74112')
,phone_array_t()
..phone1(phone_t()..Location('Home')..phone_num('918-835-3161'))
..phone2(phone_t()..Location('Mobile')..Phone_num('338-331-4463'))
..phone3(phone_t()..Location('')..phone_num(''))
..phone4(phone_t()..Location('')..phone_num(''))
,status_desc_arr()
..desc1(status_desc_t()
..Reg_id('R01')
..reg_date((select oid from time_dim_tab where time_key = '1999Q3M09'))
..marital_status('Unmarried')
..year_income(60000)
..city('Tulsa')
```

```

        ..state('Oklahoma'))
    )

```

สำหรับคำสั่งที่ใช้ในการอัปเดตข้อมูลนั้นเนื่องจากว่า IBM DB2 ยังไม่สนับสนุนการทำงานในรูปแบบข้อมูลประเภทโครงสร้างมากนัก ทำให้การอัปเดตข้อมูลในตารางที่มีข้อมูลประเภทโครงสร้างอยู่นั้น จำเป็นต้องเอาข้อมูลเก่าเก็บลงไปด้วย เพื่อไม่ให้ข้อมูลเก่าหายดังแสดงได้ในตัวอย่างข้างล่างนี้

```

UPDATE cust_dim_tab
SET status_desc = (status_desc_arr()
..desc1(status_desc_t()
    ..Reg_id('R01')
    ..reg_date((select oid from time_dim_tab where time_key = '1999Q3M09'))
    ..marital_status('Unmarried')
    ..year_income(60000)
    ..city('Tulsa')
    ..state('Oklahoma'))
..desc2(status_desc_t()
    ..Reg_id('R02')
    ..Reg_date((select oid from time_dim_tab where time_key = '2001Q1M01'))
    ..marital_status('Married')
    ..Year_income(66000)
    ..city('Skokie')
    ..state('Illinois'))
..desc3(status_desc_t()
    ..Reg_id('R03')
    ..Reg_date((select oid from time_dim_tab where time_key = '2002Q1M03'))
    ..marital_status('Married')
    ..Year_income(70000)
    ..city('Tulsa')
    ..state('Oklahoma'))
)
WHERE name..first = 'Jonathan'

```

คำสั่งสุดท้ายเป็นคำสั่งที่เกี่ยวข้องกับการลบข้อมูลในตาราง แสดงให้เห็นได้ดังนี้

```
DELETE FROM cust_dim_tab where name.first = 'Jonathan'
```

5.2.4 ตารางแสดงข้อมูลการขายสินค้า

แอททริบิวต์ของตารางแฟคนี้จะเป็นแอททริบิวต์ง่าย ๆ ไม่ซับซ้อน สำหรับการแทรกข้อมูลลงตารางนั้นเราจะใช้คำสั่งดังต่อไปนี้

```
INSERT INTO sale_fact_tab VALUES(
sale_fact_t(0),
(select oid from cust_dim_tab where name..first = 'Jonathan'),
(select oid from prod_dim_tab where model_name = 'CR-V'),
(select oid from time_dim_tab where time_key = '1999Q3M09'),
1,21550)
และสามารถอัปเดตข้อมูลได้โดยใช้คำสั่ง
UPDATE sale_fact_tab
SET time_dim = (select oid from time_dim_tab where time_key = '1999Q1M01')
WHERE time_dim->time_key = '2000Q1M02'
```

5.3 ตัวอย่างการใช้คำสั่งเอสคิวแอลในการตอบคำถาม

จุดประสงค์หลักสำคัญของหัวข้อนี้เพื่อที่จะศึกษาการตอบคำถามข้อมูลในแต่ละแบบ ในขณะที่มีการคำนวณผลลัพธ์ เราจะสามารถอ้างอิงถึงข้อมูลประเภทที่มีโครงสร้างซับซ้อนได้อย่างไร

ในหัวข้อนี้เราก็จะแสดงถึงตัวอย่างของการตอบคำถามทางด้านธุรกิจ เพื่อให้ง่ายต่อการเข้าใจเราจะแสดงตัวอย่างของการตอบคำถามประเภทง่าย ๆ ก่อน แล้วค่อย ๆ ยากขึ้นไป

ตัวอย่างแรกนั้นเราจะหาคำตอบของจำนวนรถที่ถูกขายไปในแต่ละปี เช่นปี 2000 ตัวอย่างนี้เกี่ยวข้องกับตารางแฟคและ ตารางไคเมนชันของเวลา ซึ่งอ้างอิงโดยใช้ time_dim_ref ในตารางตารางแฟค ซึ่งจะต่างกับการตอบคำถามในฐานะข้อมูลเชิงสัมพันธ์ตรงที่เราไม่จำเป็นต้องใช้การเชื่อมตาราง(Joint Table) ดังแสดงในตัวอย่าง

```
SELECT count(*)
FROM sale_fact_tab
WHERE time_dim->fiscal_year = 2000
```

คำถามต่อไปเราจะตอบคำถามที่ว่าว่ามีรถกี่คันที่ถูกผลิตในบริษัท Jeep และถูกขายได้ในปี 2000 ซึ่งจะเกี่ยวข้องกับไคเมนชัน 2 ไคเมนชันคือตารางไคเมนชันของเวลาและตารางไคเมนชันของสินค้าในตาราง Fact Table

```
SELECT count(*)
FROM sale_fact_tab
WHERE time_dim->fiscal_year = 2000 AND product_dim->manufacturer = 'Jeep'
```

ตัวอย่างต่อไปจะตอบคำถามที่ว่าจงหาชื่อบริษัทผู้ผลิตรถยนต์ที่มียอดขายรถยนต์สูงที่สุดในปี

2000

```
SELECT sf1.product_dim->manufacturer
FROM sale_fact_tab sf1
WHERE sf1.time_dim->fiscal_year = 2000
GROUP BY sf1.product_dim->manufacturer
HAVING sum(sf1.price) >= all (select sum(sf2.price)
                             FROM sale_fact_tab sf2
                             WHERE sf2.time_dim->fiscal_year = 2000
                             GROUP BY sf2.product_dim->manufacturer)
```

ถึงแม้ว่าการเชื่อมตารางจะไม่จำเป็นต่อการตอบคำถามสักเท่าไรนัก แต่สำหรับการทำซับซ้อนก็ยังคงเป็นสิ่งที่หลีกเลี่ยงไม่ได้

ตัวอย่างต่อไปเป็นการตอบคำถามที่ว่า มีรถกี่คันที่ผลิตโดยบริษัท Jeep และขายให้กับลูกค้าที่เป็นเพศชาย ในปี 2000 จากตัวอย่างนี้เราจะเห็นได้ว่าจะเกี่ยวข้องกับ 3 โดเมนชั้นของตารางแฟลคดังกล่าวข้างล่างนี้

```
SELECT count(*)
FROM sale_fact_tab
WHERE time_dim->fiscal_year = 2000 AND product_dim->manufacturer = 'Jeep'
      AND cust_dim->sex = 'M'
```

คำถามต่อไปเป็นคำถามที่ถามว่า มีลูกค้าที่เป็นเพศชายกี่คนที่ซื้อรถในระหว่างปี 2000

```
SELECT count(cust_dim->name..First)
FROM sale_fact_tab
WHERE time_dim->fiscal_year = 2000
      AND (cust_dim->status_desc..desc1..marital_status = 'Married'
           OR cust_dim->status_desc..desc2..marital_status = 'Married'
           OR cust_dim->status_desc..desc3..marital_status = 'Married')
      AND cust_dim->sex = 'M'
```


จากตัวอย่างข้างต้นจะเห็นว่าเนื่องจากว่าสถานะการสมรสของลูกค้าขึ้นอยู่กับข้อมูลประเภทโครงสร้างซับซ้อน ดังนั้นการเข้าถึงจึงต้องตรวจสอบจากสถานะทั้งหมดที่มีของลูกค้า

คำถามต่อมาเราก็จะศึกษาการเข้าถึงการตอบคำถามจากข้อมูลประเภทโครงสร้างซับซ้อนทั้งที่เป็นมัลติแวลูแอททริบิวต์และที่ไม่เป็นมัลติแวลูแอททริบิวต์ เป็นคำถามที่ถามว่า มีลูกค้าที่เป็นหญิง และมีสถานะเป็นหม้าย มีที่อยู่อยู่ที่แมนเชสเตอร์ ซื้อรถที่ผลิตโดยบริษัท JEEP ระหว่างปี 2000 จำนวนกี่คน

```
SELECT count(cust_dim->name..first)
FROM sale_fact_tab
WHERE time_dim->fiscal_year = 2000
AND cust_dim->sex = 'F'
AND (cust_dim->status_desc..desc1..marital_status = 'Divorced'
      OR cust_dim->status_desc..desc2..marital_status = 'Divorced'
      OR cust_dim->status_desc..desc3..marital_status = 'Divorced')
AND cust_dim->address..state = 'Messachusetts'
```